Document number: P2285R0
Date: 2021-01-14
Audience: EWG
Reply-to: *Andrzej Krzemieński <akrzemi1 at gmail dot com>*
Tomasz Kamiński <tomaszkam at gmail dot com>

Are default function arguments in the immediate context?

In this document we explore the question if evaluating the default function arguments should be considered the immediate context. In other words, if the following program should be well formed or not:

```
template <typename T, typename Allocator>
struct container
{
  template <std::ranges::range Range>
  explicit container(Range r, Allocator a = Allocator()) {}
};

struct Alloc
{
  Alloc() = delete;
  // ...
};

int main()
{
  constexpr bool c = std::constructible_from<container<int, Alloc>, std::vector<int>>;
  std::cout << c << std::endl;
}
```

Currently, compilers give different answers. (For the purpose of the test we replaced concept `std::constructible_from` with type trait `std::is_constructible`.) Clang and ICC fail to compile: the evaluation of the concept (trait) is ill-formed. GCC outputs 0. MSVC outputs 1.

This addresses issue CWG 2296.

Tony table:

Today

```
struct Map
{
  explicit Map(Range&&, Hash, Equal, Allocator);

  explicit Map(Range&& c, Hash h, Equal e)
   requires default_initializable<Allocator>
   : Map(c, h, e, Allocator()) {}

  explicit Map(Range&& c, Hash h)
   requires default_initializable<Equal>
         && default_initializable<Allocator>
   : Map(c, h, Equal(), Allocator()) {}

  explicit Map(Range&& c)
   requires default_initializable<Hash>
         && default_initializable<Equal>
         && default_initializable<Allocator>
   : Map(c, Hash(), Equal(), Allocator()) {}
};
```

This proposal

```
struct Map
{
  explicit Map(Range&&, Hash = Hash(), Equal = Equal(), Allo
};
```

1. Discussion

1.1. Conceptual model for default function arguments

An important question that we would like to be sorted out is what is the conceptual model behind default function arguments. Is it:

1. Additional function overloads that share implementation.
2. The "injection" of missing arguments into the function call.

Consider the following declaration

```
int f(int i, int j = make_j());
```

The first model ("additional overloads") would mean that the above is somewhat equivalent to the following two declarations:

```
int f(int i, int j);
inline int f(int i) { return f(i, make_j()); }
```

The second model ("inject missing arguments") would mean that whenever function `f` is called with one argument:

```
f(3);
```

It is replaced with:

```
f(3, make_j());
```

If we can describe the semantics of default function arguments using one of the above models, it should make it clear for the programmers how the feature behaves in different contexts.

1.1.1. Function address

Given the following code:

```
template <typename F>
void test(F);

int f(int i, int j = make_j());

int main()
{
  test(&f);
}
```

Under model "inject missing arguments" we have only one unambiguous function `f`, so the program should compile. In contrast, under model "additional overloads" expression `&f` does not refer to a single function, so the call to `test` is ambiguous. In this case the current rules clearly prefer the "inject missing arguments" model.

A related question is whether we should be able to cast a function with default arguments to a function pointer of type with fewer arguments:

```
int f(int i, int j = make_j());

using fun_ptr = int(*)(int);
fun_ptr p = f;
```

This reflects an expectation (not necessariy a correct one) that if we add a new argument to a function with a default value, any program should still compile (and have the same semantics). Under model "inject missing arguments" the above program fails to compile because there is only one `f`, and it takes two arguments. Under model "additional overloads" we have two overloads, and one of them is a perfect match. In this case, again, the current rules clearly prefer the "inject missing arguments" model.

1.1.2. Function's source location

Given the following code:

```
int f(int i, int j = std::source_location::current().line()); // line A

int main()
{
  f(1); // line B
}
```

Should the value of parameter `j` be initialized to line A or line B? Model "inject missing parameters" suggests line B, and this is what the Standard requires.

1.1.3. How the names are looked up

Consider the following example:

```
// included from header file:

const int width = 4;

void foo(int val, int w = width) { std::cout << std::string(w, '=') << val << " "; }

// in a cpp file:

int main()
{
  for (int width : {1, 2, 3})
    foo(width);
}
```

In model "additional overloads" this program outputs `"====1 ====2 ====3 "`. In model "inject missing parameters" this program outputs `"=1 ==2 ===3 "` because what is injected is the nested `width`: the one declared inside the `for`-loop.

1.1.4. Being in immediate context

Finally, the question of being in immediate context. In model "inject missing parameters" the expressions injected in the call site clearly become part of immediate context. On the other hand, in mode "additional overloads" the expressions for computing missing parameters are part of function bodies and are not part of immediate context.

1.2. Consistency with default member initializers

We believe that whatever solution is applied to default function arguments, it should also be applied to defult member initializers. This is for consistency rasons, as the two feturesare similar:

```
template <typename T>
struct S
{
  T x;
  T y = T{};

  // explicit S(T x, T y = T{}) : x{x} y{y} {}
};

struct NoDefault
{
  explicit NoDefault(int) {}
};

int main()
{
  std::cout << std::constructible_from<C, NoDefault> << std::endl;
}
```

The answer to the question whether this program compiles and what value is printed shouls not change if we uncomment the constructor in class `S`.

Currently, compilers vary in whether they treat default member initializers as an immediate context. Consider the following example:

```
struct NoDefault
{
  explicit NoDefault(int) {}
};

template<typename T>
struct S
{
  int x;
  T y = T{};
};

template <typename T>
int convertible(...) { return 0; }

template <typename T>
auto convertible(int) -> decltype(S<T>{2}.x) { return 1; }

int main()
{
   std::cout << convertible<NoDefault>(1) << std::endl;
}
```

Currently, GCC and MSVC output 0, Clang and ICC fail to compile.

1.3. Applicability of default function arguments

Is there sufficient motivation to add a requirement that default function arguments be in the immediate context of the function call? We believe there is. With C++20's concepts quetying for types' properties and designing conditional interfaces will become more common and less expert-only. We already have features like `explicit(bool)` introduced primarily for rendering non-uniform instantations of class templates based on the properties of template arguments.

Now, with the addition of ranges library to C++, we may want to introduce container constructors that take ranges as arguments. In practice, for a container like `unordered_map` we will need variations that take a hash, a comparator, an allocator. This could be handled by one constructor declaration with default function arguments:

```
template <class Key, class Hash = /**/, class KeyEqual = /**/, class Allocator = /**/ >
class unordered_set;
{
  explicit unordered_set(range auto&&, Hash = Hash(), KeyEqual = KeyEqual(), Allocator = Allocator());
};
```

But will `constructible_from<unordered_map<int>, MyIntHash>, std::vector<int>>` give the true answer?

Unless default function arguments are in the immediate context, if we want concept `std::constructible_from` to return the true answer, we will have to provide a number of constructors:

```
template <class Key, class Hash = /**/, class KeyEqual = /**/, class Allocator = /**/ >
class unordered_set;
{
  explicit unordered_set(range auto&&, Hash, KeyEqual, Allocator);

  explicit unordered_set(range auto&& c, Hash h, KeyEqual e)
   requires default_initializable<Allocator>
   : unordered_set(c, h, e, Allocator()) {}

  explicit unordered_set(range auto&& c, Hash h)
   requires default_initializable<KeyEqual>
         && default_initializable<Allocator>
   : unordered_set(c, h, KeyEqual(), Allocator()) {}
```

```
    explicit unordered_set(range auto&& c)
     requires default_initializable<Hash>
           && default_initializable<KeyEqual>
           && default_initializable<Allocator>
     : unordered_set(c, Hash(), KeyEqual(), Allocator()) {}

  };
```

But if we did so, one could ask, what are default function argument for, if the Standard Library cannot use them?

Alternatively, we can ask, is the above case with `unordered_map` something that default function arguments are supposed to handle?

2. Our Recommendation

We recommend that the model for default function arguments as well as for default member initializers is "inject missing arguments" except for one aspect: name lookup is performed as if it was inside the function body (as in "additional overloads").

3. Wording

TBD.

4. Acknowledgments

Tim Song indicated the full scale of the problem with default function arguments, which motivated the scope and shape of this paper.

We are grateful to Hubert Tong and David Vandevoorde for their valuable input.

5. References

- [P0348R0] — Andrzej Krzemieński, "Validity testing issues" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0348r0.html).
- [P1073R2] — Richard Smith, Andrew Sutton, Daveed Vandevoorde, "Immediate functions" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1073r2.html).